HealthTrack Phase 3: Security and Performance Review

Shawn Miller | MSCS-699-B01 | University of the Cumberlands
Instructor: Michael Solomon | November 8, 2026

Project overview

Phase 3 turns the HealthTrack design into a working FastAPI backend for a small, consent-based hypertension-monitoring pilot. The implemented tables center on patients, blood pressure and pulse readings, and daily activity records. Supporting accounts, care-team assignments, and audit events allow the project to demonstrate who can access each record. This is a local school project using fictional patients, not a system ready for clinical use.

Security measures implemented

The API requires a signed bearer token for protected routes. Demo passwords are stored as salted PBKDF2 hashes rather than plain text. Tokens expire after 30 minutes, and the service checks whether accounts are active. Role and ownership checks limit patient records to their owners, while nurses and clinicians can only access patients assigned to them. Administrators can manage enrollment and assignments but cannot access vital-sign or activity endpoints.

Input checks reject underage enrollment, invalid blood pressure values, out-of-range pulse or activity entries, future-dated records, and missing timezones for readings. The database also enforces key value ranges, foreign keys, and unique daily activity records. New measurements require active enrollment and monitoring consent. Generic audit records capture key access and changes without storing measurement values in the audit text. Errors return clear HTTP status codes, and database integrity conflicts trigger a rollback.

Important security limitations

The current authentication uses a local demonstration JWT issuer rather than the trusted OpenID Connect provider proposed in Phase 2. MFA, request rate limiting, secure hosting and TLS setup, managed encryption, recovery testing, immutable reading revisions, complete denied-request audits, and a clinic-level privacy assessment remain unfinished. The DELETE endpoints were included to meet the CRUD requirement, but a real clinic would need an approved records-retention process instead of unrestricted hard deletion. No real patient data should be entered into this version.

Testing and performance results

All 33 automated tests passed, with no failed tests and 94% total application statement coverage. Tests covered model constraints, invalid inputs, authorization, consent, CRUD operations, date filters, missing records, duplicate activity entries, and audit creation. Both Alembic migrations were applied successfully to a local SQLite database.

Local database benchmark. A synthetic SQLite dataset included 5,000 vital records and 500 activity records. Across 200 date-filtered queries returning up to 50 readings, median retrieval time was 0.384 ms, 95th-percentile time was 0.567 ms, and maximum observed time was 3.835 ms. These measurements reflect one machine and a direct database microbenchmark, not API throughput or production concurrency.

Conclusion and next steps

The prototype meets the main Phase 3 implementation goals: organized SQLAlchemy models, a reusable data-access layer, documented FastAPI endpoints, migration scripts, and testable access rules. Next steps are to replace demo authentication with external identity management and MFA, enforce full clinic isolation and retention rules, complete revision and alert workflows, and test with a production-like database and concurrent API requests. These improvements would be necessary before considering any clinical pilot.